- Some functions assume that the string is encoded in some (any) single-byte encoding, but they do not need to interpret those bytes as specific characters. This is case of, for instance, substr(), strpos(), strlen() or strcmp(). Another way to think of these functions is that operate on memory buffers, i.e., they work with bytes and byte offsets.
- Other functions are passed the encoding of the string, possibly they also assume a default if no such information is given. This is the case of htmlentities() and the majority of the functions in the mbstringextension.
- Others use the current locale (see setlocale()), but operate byte-by-byte. This is the case of strcasecmp(),strtoupper() and ucfirst(). This means they can be used only with single-byte encodings, as long as the encoding is matched by the locale. For instance *strtoupper("á")* may return *"Á"* if the locale is correctly set and *á* is encoded with a single byte. If it is encoded in UTF-8, the correct result will not be returned and the resulting string may or may not be returned corrupted, depending on the current locale.
- Finally, they may just assume the string is using a specific encoding, usually UTF-8. This is the case of most functions in the intl extension and in the PCRE extension (in the last case, only when the *u* modifier is used). Although this is due to their special purpose, the function utf8_decode() assumes a UTF-8 encoding and the function utf8_encode() assumes an ISO-8859-1 encoding.

Ultimately, this means writing correct programs using Unicode depends on carefully avoiding functions that will not work and that most likely will corrupt the data and using instead the functions that do behave correctly, generally from the intl and mbstring extensions. However, using functions that can handle Unicode encodings is just the beginning. No matter the functions the language provides, it is essential to know the Unicode specification. For instance, a program that assumes there is only uppercase and lowercase is making a wrong assumption.